FAST: FPGA Acceleration of Fully Homomorphic Encryption with Efficient Bootstrapping

Zhihan Xu
University of Southern California
Los Angeles, USA
zhihanxu@usc.edu

Tian Ye
University of Southern California
Los Angeles, USA
tye69227@usc.edu

Rajgopal Kannan
DEVCOM Army Research Office
Los Angeles, USA
rajgopal.kannan.civ@army.mil

Viktor K. Prasanna
University of Southern California
Los Angeles, USA
prasanna@usc.edu

Abstract

Bootstrapping is a critical operation in Fully Homomorphic Encryption (FHE) for privacy-preserving computation. Due to its significant computational overhead, accelerating bootstrapping is crucial for practical FHE applications involving deep evaluation circuits.

In this paper, we introduce FAST, an FPGA-based accelerator for efficient FHE bootstrapping. We propose novel datapath optimizations for two key operations in bootstrapping: homomorphic linear transformation (HLT) and polynomial evaluation. Our memory-efficient datapath designed for HLT significantly reduces off-chip ciphertext access. We also speed up the polynomial evaluation process by reducing the number of required HE operations. We conduct an in-depth analysis of the Advanced Bootstrapping Algorithm (ABA) and highlight its computational advantages. FAST is the first accelerator to support ABA, demonstrating significant speedup for bootstrapping. In addition, we develop a novel versatile permutation circuit to handle diverse permutation patterns in FHE, achieving high throughput and efficient resource utilization. Compared with the state-of-the-art (SOTA) GPU and FPGA designs, FAST achieves 8.84× and 5.89× speedups for bootstrapping, respectively. As illustrative examples of deep FHE applications, we show that FAST delivers over 20× speedup for logistic regression training compared with the SOTA GPU implementation and outperforms the SOTA FPGA design by 1.43× for ResNet-20 inference.

CCS Concepts

• **Hardware → Hardware accelerators**; • **Computer systems organization → Reconfigurable computing**; • **Security and privacy → Cryptography**.

Keywords

FPGA Acceleration, Fully Homomorphic Encryption, CKKS, Efficient Bootstrapping, Privacy-preserving Computation

This work is licensed under a Creative Commons Attribution International 4.0 License.

FPGA '25, February 27-March 1, 2025, Monterey, CA, USA

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1396-5/25/02
https://doi.org/10.1145/3706628.3708879

ACM Reference Format:
Zhihan Xu, Tian Ye, Rajgopal Kannan, and Viktor K. Prasanna. 2025. FAST: FPGA Acceleration of Fully Homomorphic Encryption with Efficient Bootstrapping. In *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA '25), February 27-March 1, 2025, Monterey, CA, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3706628.3708879

1 Introduction

Homomorphic Encryption (HE) [23] is a promising technique for preserving data privacy in cloud computing, enabling direct computations on encrypted data. Its applications span various fields, such as privacy-preserving machine learning [26, 43, 50, 51], secure database management [57, 65], confidential genome analysis [37, 66], among others. HE schemes are based on learning with errors, utilizing a noise term to ensure privacy. However, as HE operations progress, the noise accumulates and can eventually become overwhelming [52]. If the noise exceeds a certain threshold, the underlying message cannot be correctly decrypted [46]. To address this, bootstrapping is periodically performed to reduce the noise, enabling an unlimited number of HE operations [23]. HE schemes that support bootstrapping are known as Fully Homomorphic Encryption (FHE). Among the various FHE schemes, we focus on the CKKS scheme [18], which has gained popularity due to its support for real arithmetic for Machine Learning (ML) tasks [53].

Compared to other basic HE operations, bootstrapping remains the most computationally expensive [58], characterized by low arithmetic intensity [20], and significant memory traffic [34]. It is the primary bottleneck of FHE [33], dominating the execution of complex workloads, particularly when the evaluation circuit is deep [40], e.g., in deep learning [42]. One bootstrapping operation takes tens to hundreds of seconds on a modern CPU [12, 13, 28], consuming up to 95% of the total application execution time [22, 35]. Consequently, efficiently performing bootstrapping is critical for practical FHE applications.

The complex dataflow and extensive data transfers in bootstrapping make it extremely time-consuming. To address the computation and memory overhead of bootstrappable FHE, various algorithmic optimizations and acceleration efforts have been proposed [13, 20, 28, 34, 36]. However, CPU/GPU-based approaches [24, 34, 48] achieve limited performance due to their cache-based hierarchy

and lack of explicit control over data movement. Moreover, complex data permutations in HE cannot be efficiently handled by CPU/GPU. ASIC solutions [21, 36, 38] can offer promising performance by leveraging many compute units and large on-chip memory. However, given that bootstrapping algorithms are evolving, the significant nonrecurring engineering costs associated with ASIC implementations cannot be ignored [2]. Benefiting from the configurable on-chip SRAM memory hierarchy and customizable compute cores, FPGAs can be tailored to increase on-chip data reuse and also achieve fast modular operations. Major FPGA vendors [8, 32] also incorporate High Bandwidth Memory (HBM) to address the limitations of DRAM bandwidth. The latest FPGAs can provide up to 820 GB/s bandwidth and 32 GB HBM storage [3], making them highly suitable for accelerating memory-intensive tasks such as bootstrapping.

State-of-the-art (SOTA) FPGA implementations of bootstrappable FHE outperform general-purpose processors in bootstrapping performance. FAB [2] is the first FPGA design to support CKKS bootstrapping with a carefully optimized datapath that fully utilizes all available on-chip storage. Poseidon [62] optimizes several subroutines in FHE and develops hardware-friendly data permutation. However, its permutation circuit is not reusable for diverse permutation patterns involved in bootstrapping. Despite these advances, neither design specifically optimizes the bootstrapping dataflow, resulting in low data reuse due to the coarse-grained operations in bootstrapping. Furthermore, these accelerators do not leverage recent advances in bootstrapping algorithms; this leads to higher computation and memory costs compared to the Advanced Bootstrapping Algorithm (ABA) [10], a SOTA bootstrapping algorithm.

In this paper, we propose FAST, an FPGA-based FHE accelerator demonstrating superior CKKS bootstrapping performance. FAST is the first accelerator to support ABA. It leverages our novel datapath optimizations for two key operations in bootstrapping: homomorphic linear transformation (HLT) and polynomial evaluation. In summary, our main contributions are:

- We propose a memory-efficient datapath for HLT (ME-HLT) through algorithm-architecture co-design, improving the fine-grained on-chip ciphertext reuse via subroutine fusion and significantly reducing off-chip ciphertext transfers. We also optimize the polynomial evaluation process, reducing the number of HE operations and computation requirements.
- We develop a novel versatile permutation circuit capable of handling a variety of permutation patterns in FHE, achieving high throughput, efficient resource utilization, and enabling parallel processing alongside computation.
- We analyze the computational requirements of ABA and design FAST, the first accelerator to support ABA. It reduces both computational requirements and off-chip data transfer so as to realize low-latency end-to-end FHE applications.
- We implement FAST on AMD Alveo U280, achieving bootstrapping speedups of 8.84× and 5.89× over SOTA GPU and FPGA designs, respectively, based on amortized multiplication per slot. For complex FHE applications, FAST delivers over 20× speedup for logistic regression training compared to the GPU and outperforms the SOTA FPGA by 1.43× for ResNet-20 inference.

Table 1: CKKS Parameters

Param.	Description
N	# of coefficients in a ciphertext polynomial
L	Maximum level of ciphertext
Q or Q_L	$= \prod_{i=0}^{L} q_i$, Ciphertext initial modulus
Q_ℓ	$= \prod_{i=0}^{\ell} q_i$, Ciphertext current modulus at level ℓ
k	# of moduli in P
P	$= \prod_{i=0}^{k-1} p_i$, Auxiliary modulus for `Keyswitch`
dnum	RNS decomposition number. Max # of digits
α	$= (L+1)/\mathsf{dnum}$, # of limbs in one digit
β	$= \lceil \ell + 1 \rceil / \alpha$, # of digits in ciphertext

Table 2: HE Subroutines

Subroutine	Description
NTT/INTT	Number theoretic transform and its inversion
BaseConv	RNS basis conversion
ModUp	Raise the ct modulus; increase # of ct limbs
ModDown	Decrease the ct modulus and # of limbs
KeySwitch	Revert the underlying decryption key
Decomp	RNS basis decomposition for KeySwitch
InnerProd	Hadamard product with key or plaintexts
Automorph	Polynomial permutation, denoted by ψ

2 Background

In this section, we briefly review the CKKS scheme [18] and FHE operations. The parameters used are summarized in Table 1.

2.1 FHE and the RNS-CKKS

CKKS supports the encryption of complex fixed-point vectors. A message vector $\mathbf{m} \in \mathbb{C}^{N/2}$ is encoded into a plaintext (pt) polynomial $\mathrm{P}_{\mathbf{m}}$ in $\mathcal{R}_Q = \mathbb{Z}_Q[X]/(X^N+1)$. The pt is then encrypted with a secret key into a ciphertext (ct) $[\![\mathbf{m}]\!] := (\mathbf{a}, \mathbf{b})$ in $\mathcal{R}_Q^2$ with two polynomials. The ct coefficients are integers with modulus Q.

The full modulus Q is often on the order of thousands of bits. To compute on such large ct coefficients, the Residual Number System (RNS) can be employed. In RNS, Q is represented as the product of the coprime moduli $Q = \prod_{i=0}^{L} q_i$, where each q_i is less than a machine word (i.e., 64 bits). The CKKS with the RNS technique is called RNS-CKKS [11, 17]. In RNS-CKKS, each ciphertext polynomial is represented with multiple limbs, i.e., $[\mathbf{a}]_Q \mapsto [\mathbf{a}]_{q_0}, \cdots, [\mathbf{a}]_{q_L}$. Each limb is a subpolynomial with smaller coefficients and a unique modulus. The number of limbs in one ct equals $(L+1)$.

Subroutines: We define the subroutines to compose HE operations, summarized in Table 2. NTT is used for efficient polynomial multiplication, which reduces the complexity from $O(N^2)$ to $O(N \log N)$ by transforming the polynomials into the evaluation domain, where multiplication can be performed point-wise [44]. To avoid unnecessary NTTs and its inversion INTTs, polynomials are typically kept in the evaluation domain [34, 36, 38]. BaseConv allows the conversion of the RNS representation of a polynomial from one basis to another, e.g., $[\mathbf{a}]_Q \mapsto [\mathbf{a}]_P$, which is the foundation for two key subroutines in HE, namely ModUp and ModDown [17]. As BaseConv cannot operate in the evaluation domain, INTT is applied before BaseConv and NTT is applied afterward.

Algorithm 1 KeySwitch($[\mathbf{a}]_{Q_\ell}, \mathbf{ksk}$)

1: $\overrightarrow{\mathbf{a}} = (\mathbf{a}_j)_{j\in[0,\beta)} \leftarrow \mathsf{Decomp}(\mathbf{a})$ ▷ $\mathcal{R}_{Q_\ell}$
2: $\overrightarrow{\mathbf{a}} = ([\hat{\mathbf{a}}_j]_{PQ_\ell})_{j\in[0,\beta)} \leftarrow \mathsf{ModUp}(\mathbf{a}_j)$ **for** $j \in [0,\beta)$ ▷ $\mathcal{R}^{\beta}_{PQ_\ell}$
3: $(\hat{\mathbf{c}}_0, \hat{\mathbf{c}}_1) \leftarrow \mathsf{KSKInnerProd}(\overrightarrow{\mathbf{a}}, \mathbf{ksk})$ ▷ $\mathcal{R}^{2}_{PQ_\ell}$
4: $(\mathbf{c}_0, \mathbf{c}_1) \leftarrow \mathsf{ModDown}(\hat{\mathbf{c}}_0), \mathsf{ModDown}(\hat{\mathbf{c}}_1)$ ▷ $\mathcal{R}^{2}_{Q_\ell}$
5: **return** $(\mathbf{c}_0, \mathbf{c}_1)$

Table 3: CKKS operations [36]

Operation	Output
CAdd($[\![\mathbf{m}]\!], c$)	$[\![\mathbf{m}+\mathbf{c}]\!] = (\mathbf{a}, \mathbf{b}+\mathbf{c})$
CMult($[\![\mathbf{m}]\!], c$)	$[\![\mathbf{m}\cdot\mathbf{c}]\!] = (\mathbf{a}, \mathbf{b}\cdot\mathbf{c})$
PAdd($[\![\mathbf{m}]\!], \mathrm{P}_{\mathbf{m}'}$)	$[\![\mathbf{m}+\mathbf{m}']\!] = (\mathbf{a}, \mathbf{b}+\mathrm{P}_{\mathbf{m}'})$
PMult($[\![\mathbf{m}]\!], \mathrm{P}_{\mathbf{m}'}$)	$[\![\mathbf{m}\cdot\mathbf{m}']\!] = (\mathbf{a}\cdot\mathrm{P}_{\mathbf{m}'}, \mathbf{b}\cdot\mathrm{P}_{\mathbf{m}'})$
HAdd($[\![\mathbf{m}]\!], [\![\mathbf{m}']\!]$)	$[\![\mathbf{m}+\mathbf{m}']\!] = (\mathbf{a}+\mathbf{a}', \mathbf{b}+\mathbf{b}')$
HMult($[\![\mathbf{m}]\!], [\![\mathbf{m}']\!], \mathbf{ksk}$)	$[\![\mathbf{m}\cdot\mathbf{m}']\!] = \mathsf{KeySwitch}(\mathbf{b}\cdot\mathbf{b}', \mathbf{ksk}) + (\mathbf{a}\cdot\mathbf{a}', \mathbf{a}\cdot\mathbf{b}'+\mathbf{b}\cdot\mathbf{a}')$
Rotate($[\![\mathbf{m}]\!], d, \mathbf{ksk}$)	$[\![\mathbf{m}\ll d]\!] = \mathsf{KeySwitch}(\psi_d(\mathbf{b}), \mathbf{ksk}) + (\psi_d(\mathbf{a}), \mathbf{0})$
Conjugate($[\![\mathbf{m}]\!], \mathbf{ksk}$)	$[\![\overline{\mathbf{m}}]\!]$ (implemented as Rotate)
Rescale($[\![\mathbf{m}]\!]$)	$[\![\Delta^{-1}\cdot\mathbf{m}]\!] = (\Delta^{-1}\cdot\mathbf{a}, \Delta^{-1}\cdot\mathbf{b})$

A few HE operations (i.e., HMult, Rotate, Conjugate) require KeySwitch, shown in Algorithm 1, as the underlying decryption key changes after these operations. The subroutine relies on a key-switching key $\mathbf{ksk} \in \mathcal{R}^{2\times\beta}_{PQ_\ell}$, which is a $2\times\beta$ matrix of polynomials. RNS basis decomposition Decomp splits the ct limbs into β digits (or, groups) [28, 34]. The maximum value of β is dnum. Each digit comprises α limbs, except for the last digit, which may contain $\leq \alpha$ limbs. After Decomp, ModUp is applied to support KSKInnerprod, increasing the number of limbs of each digit to $\ell + k + 1$.

The Automorph subroutine enables the permutations required for Rotate and Conjugate operations. In Rotate, the circular left shifting the underlying message by d is equivalent to the permutation on ct by the coefficient index mapping pattern: $\psi_d : i \mapsto i \cdot 5^d \bmod N$ through Automorph. Using a clever encoding method [19], Conjugate is implemented in the same way as the Rotate operation.

Operations: The FHE operations, including ct-constant, ct-pt, and ct-ct operations, are summarized in Table 3. During encoding, the original message $\mathbf{m}$ is scaled by a factor Δ. After each ct-pt/ct-ct multiplication (i.e., PMult, HMult), the scale of the result becomes Δ^2. To reduce this scale, Rescale is performed to transform a level ℓ encryption of $\mathbf{m}$ into a level $\ell-1$ encryption of $q_\ell^{-1}\cdot\mathbf{m}$ [17]. Here, a level ℓ encryption means that the ct has modulus of $Q_\ell = \prod_{i=0}^{\ell} q_i$ and has $(\ell+1)$ limbs under RNS. Rescale is equivalent to $\mathsf{ModDown}_{Q_\ell \to Q_{\ell-1}}$ [1], discarding the last prime modulus q_ℓ.

2.2 Bootstrapping

As discussed, the Rescale operation is necessary after HMult and PMult, consuming one level of the ct. After several (ℓ) ct-ct/ct-pt multiplications, the level of the ct will eventually reduce to 0, leaving only the base limb with modulus q_0. No further multiplications can be performed. To enable further computations on a ct, a complex operation known as bootstrapping is required [12, 28]. This operation raises the modulus (or levels) while preserving the correct structure of the ct. Bootstrapping is a significant bottleneck in FHE, which is the main focus of this paper.

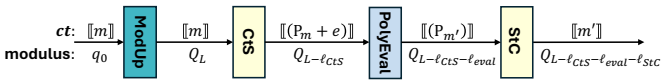

Figure 1: Traditional CKKS bootstrapping datapath

Algorithm 2 BSGS algorithm for HLT (one *fftiter*)

Input: $[\![\mathbf{m}]\!]$, $r = r_1 \cdot r_2$, Precomputed plaintext diagonals: $\{\mathbf{M}_i\}_{i=0}^{r-1}$
Output: $[\![\mathbf{m}']\!]$ initialized as zero

1: **for** i from 0 to $r_1 - 1$ **do** ▷ Baby Step: Rotate ct
2: $[\![\mathbf{m}_i]\!] \leftarrow \mathsf{Rotate}([\![\mathbf{m}]\!], i\cdot d, \mathbf{ksk}_i)$
3: **for** j from 0 to $r_2 - 1$ **do** ▷ Giant Step: Rotate partial sum
4: $[\![\mathbf{m}'']\!] \leftarrow (0,0)$
5: **for** i from 0 to $r_1 - 1$ **do** ▷ DiagInnerProd
6: $[\![\mathbf{m}'']\!] \leftarrow \mathsf{HAdd}([\![\mathbf{m}'']\!], \mathsf{PMult}([\![\mathbf{m}_i]\!], \mathbf{M}_{r_1\cdot j+i}))$
7: $[\![\mathbf{m}']\!] \leftarrow \mathsf{HAdd}([\![\mathbf{m}']\!], \mathsf{Rotate}([\![\mathbf{m}'']\!], r_1\cdot j\cdot d, \mathbf{ksk}_{r_1\cdot j})$
8: **return** Rescale($[\![\mathbf{m}']\!]$)

The traditional CKKS bootstrapping shown in Fig. 1 consists of four steps: ModUp, Coefficients-to-Slots (CtS), Polynomial Evaluation (PolyEval) and Slots-to-Coefficients (StC). ModUp raises the ct modulus from q_0 to Q_L. However, this step leads to an error term (e) in encrypted pt, and the subsequent three steps remove the error by performing modular reduction, which is approximated by PolyEval. It is noteworthy that bootstrapping itself also consumes ct levels. We denote ℓ_{CtS}, ℓ_{eval}, and ℓ_{StC} as the level consumption at each step, with the resulting modulus after each step shown in Fig. 1.

2.2.1 Homomorphic Linear Transformation (HLT). CtS and StC are referred to as HLT, which evaluate the decoding (i.e. IDFT) and encoding (i.e., DFT) homomorphically. The operation involves a series of pt-matrix ct-vector multiplication (PtMatCtVecMult), where the matrix is the encoded (I)DFT matrix.

The HLT operation requires a significant amount of off-chip data transfer, which consumes the majority of time during bootstrapping [12]. To reduce both computational and memory overheads, the (I)DFT matrix is decomposed into *fftiter* (i.e., $\log_{rdx}\frac{N}{2}$) sparse diagonal submatrices with radix *rdx*, each containing r (i.e., $2\cdot rdx - 1$) diagonals [25]. Each iteration (*fftiter*) is a PtMatCtVecMult. This decomposition transforms the operation into the sum of Hadamard products of these diagonals with the ct. To align with the r diagonals, r Rotate operations are required. A widely used algorithm for HLT is Baby-Step Giant-Step (BSGS) algorithm [12], which reduces the number of Rotate operations from $O(r)$ to $O(r_1 + r_2)$ (i.e., $O(\sqrt{r})$), shown in Algorithm 2. After each iteration (*fftiter*), a Rescale operation is performed, which consumes one ct level. While a larger *fftiter* can reduce the number of pt diagonals and the required **ksk** for Rotate, it also consumes more levels ($\ell_{CtS} = \ell_{StC} = fftiter$), ultimately decreasing the available ct levels after bootstrapping.

Algorithm 3 BSGS algorithm for polynomial evaluation

Input: n: degree of p; $m := \lceil \log_2 n \rceil$; $h := \lfloor m/2 \rfloor$; Coefficients of polynomial $p = \sum_{k=0}^{n} c_k T_k$
Output: The evaluation of p

1: Homomorphic evaluate $T_2, T_3, \cdots, T_{2^h-1}, T_{2^h}$ with Eq. 1
2: Homomorphic evaluate $T_{2^{h+1}}, T_{2^{h+2}}, \cdots, T_{2^{m-2}}, T_{2^{m-1}}$ with Eq. 1
3: Represent $p = p_{(0,0)} \cdot T_{2^{m-1}} + p_{(0,1)}$, where $p_{(0,0)}, p_{(0,1)}$ are polynomials of degree $< 2^{m-1}$ ▷ Binary tree: top node
4: Represent $p_{(0,0)}$ and $p_{(0,1)}$ recursively by repeating 3 until the degrees of $p_{(i,j)} < 2^h, 0 \le i < m-h, 0 \le j < 2^{i+1}$
5: Bottom-up: evaluate the lowest level polynomials $p_{(m-h-1,j)}$ with $T_0, \cdots, T_{2^h}$ and corresponding coefficients ▷ BS
6: Evaluate upper levels with representations in 3 & 4 ▷ GS
7: **return** $p = p_{(0,0)} \cdot T_{2^{m-1}} + p_{(0,1)}$

2.2.2 Homomorphic Polynomial Evaluation. The homomorphic evaluation of modular reduction involves two key steps. First, as noted in [16], the modulo q reduction function to remove the error term behaves as an identity near zero and is periodic with a period of q. This behavior can be approximated by a scaled sine function: $F(t) = (q/2\pi) \cdot \sin(2\pi t/q)$. Second, since the sine function is not directly available in FHE operations, it can be approximated by a Chebyshev interpolant p, as proposed in [13].

Here, we briefly introduce the widely used BSGS algorithm[1] [12, 28] to evaluate the Chebyshev interpolant. The Chebyshev polynomials follow this recurrence relation:

$$T_0(t) = 1, T_1(t) = t \tag{1a}$$

$$T_{a+b}(t) = 2T_a(t)T_b(t) - T_{|a-b|}(t) \tag{1b}$$

The BSGS algorithm for PolyEval in Algorithm 3 helps to reduce the number of required HMult and only consumes $\lceil \log n \rceil$ depth, where n denotes the polynomial degree of the Chebyshev interpolant used for sine approximation.

2.2.3 Advanced Bootstrapping Algorithm (ABA). The ABA [10] shown in Algorithm 4 is an alternative to the traditional bootstrapping algorithm shown in Fig. 1, with three main modifications. First, StC is moved from the final step to the first step of the datapath. This reordering was initially introduced in slim bootstrapping [14] and later adopted by [34]. Its benefits on computation will be discussed in Sec. 3.3. Additionally, the input ct is multiplied by an integer c before bootstrapping and divided by c afterward. This adjustment increases the CKKS precision during bootstrapping [10]. Lastly, an additional Conjugate is required.

3 Methodology

In this section, we introduce the proposed optimizations for bootstrapping and analyze the computation requirement of ABA.

3.1 Memory Efficient Homomorphic Linear Transformation (ME-HLT)

The BSGS Algorithm 2 benefits from reducing the number of rotations. However, storing all intermediate ct requires large on-chip memory, which leads to memory inefficiency and bottlenecks on CPU/GPU/FPGA platforms. Thus, we design a bandwidth-efficient HLT datapath suitable for SRAM-limited devices, e.g., FPGAs.

[1]Both HLT and polynomial evaluation can adopt the generic BSGS idea.

Algorithm 4 Advanced Bootstrapping Algorithm

Input: $[\![\mathbf{m}]\!] \in \mathcal{R}^2_{Q_{\ell_{StC}}}$, parameter: $c \in \mathbb{Z}$
Output: $[\![\mathbf{m'}]\!] \in \mathcal{R}^2_{Q_{L-\ell_{CtS}-\ell_{eval}}}$

1: $[\![\mathbf{m'}]\!] \leftarrow \texttt{CMult}([\![\mathbf{m}]\!], c)$
2: $[\![\mathbf{m'}]\!] \leftarrow \text{CtS} \circ \text{ModUp} \circ \text{StC}([\![\mathbf{m'}]\!])$
3: $[\![\mathbf{m'}]\!] \leftarrow (\text{Conjugate}([\![\mathbf{m'}]\!]) + [\![\mathbf{m'}]\!])/2$
4: $[\![\mathbf{m'}]\!] \leftarrow \text{PolyEval}([\![\mathbf{m'}]\!])$
5: **return** $\texttt{CMult}([\![\mathbf{m'}]\!], 1/c)$

3.1.1 Algorithmic Analysis. The byte per coefficient $\mathcal{B}_{coeff}$ can be defined as $(\log q)/8$. The size per limb $\mathcal{B}_{limb}$ is $N \cdot \mathcal{B}_{coeff}$. The sizes of a ct and a key-switching key at level ℓ can be given by:

$$\mathcal{B}^{\ell}_{ct} = 2 \cdot (\ell + 1) \cdot \mathcal{B}_{limb} \tag{2}$$

$$\mathcal{B}^{\ell}_{\mathbf{ksk}} = 2 \cdot (\ell + 1) \cdot \beta \cdot \mathcal{B}_{limb} \tag{3}$$

There are two rotation loops in BSGS Algorithm 2, with r_1 and r_2 rotation iterations (operations) for baby steps (BS) and giant steps (GS), respectively. Each Rotate operation requires one KeySwitch that necessities Decomp, ModUp and ModDown. A widely adopted optimization in HLT is double-hoisting [12], which amortizes several HE subroutines (i.e., Decomp, ModUp, ModDown) costs over multiple rotations with the same input ct. For example, as all r_1 Rotate operations in the BS rotation loop (Line 2, Algorithm 2) share the same input ct, their Decomp and ModUp (Line 1-2, Algorithm 1) can be hoisted ahead of the BS rotation loop and execute only once. However, these subroutines are still required for each of the r_2 Rotate operations in the GS rotation loop, as all r_2 Rotate operations (Line 7, Algorithm 2) have different input ct. Therefore, the double-hoisting optimization can reduce the number of these subroutines from $(r_1 + r_2)$ to $(r_2 + 1)$.

At the beginning of BSGS with double-hoisting (readers can refer to Line 1-6 of Algorithm 6 in [12]), $(r_1 + \beta)$ intermediate ct with modulus PQ_ℓ are generated, which significantly increases the on-chip memory requirement. To store all intermediate ct and eliminate off-chip transfers, the on-chip memory required for the BS iterations must be at least $(r_1 + \beta) \cdot \mathcal{B}^{\ell+k+1}_{ct}$. Under a typical parameter setup[2] used in [2], the required on-chip memory for the BS iterations amounts to approximately 300 MB. As a result, storing all intermediate ct in on-chip memory is impractical on modern CPU, GPU, and FPGA platforms due to the limited size of SRAM. Instead, these ct require off-chip memory transfer during the GS iterations, leading to significant memory bottlenecks on platforms with limited SRAM. Specifically, data traffic is at least $2 \cdot (r_1 + r_2 \cdot r_1) \cdot \mathcal{B}^{\ell+k+1}_{ct}$ for reading and writing ct during two rotation loops, amounting to over 2 GB. Given a total of $2 \cdot \mathit{fftiter}$ iterations of the HLT, several tens of gigabytes of ct must be communicated with the off-chip memory, leading to significant memory inefficiency.

[2]$N = 2^{16}$, dnum $= 3, L = 23, k = 8, \mathit{fftiter} = 4$

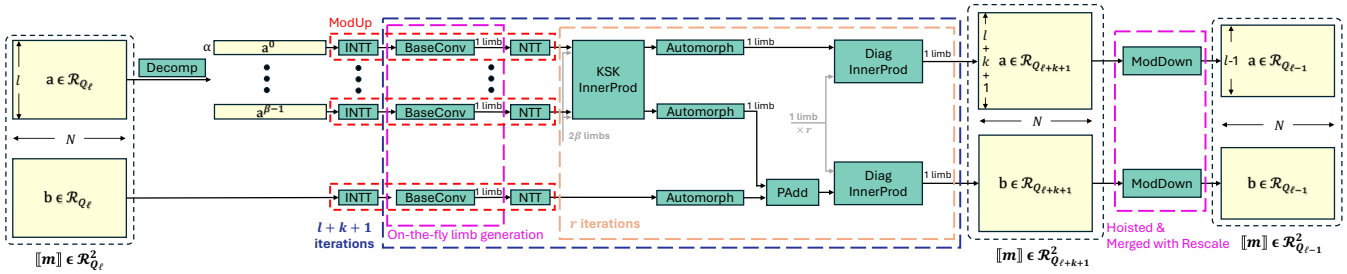

Figure 2: Memory Efficient Homomorphic Linear Transformation (ME-HLT) per *fftiter*

Algorithm 5 HLT after collapsing BSGS w. hoisting (one *fftiter*)

Input: $[\![\mathbf{m}]\!] := (\mathbf{a}, \mathbf{b})$, Precomputed plaintext diagonals: $\{\mathbf{M}_i\}_{i=0}^{r-1}$
Output: $[\![\mathbf{m}']\!] := (\mathbf{a}', \mathbf{b}')$ initialized as 0

1: $(\mathbf{a}_j)_{j\in[0,\beta)} \leftarrow \mathsf{Decomp}(\mathbf{a})$ ▷ $\mathcal{R}^2_{Q_\ell}$
2: $(\hat{\mathbf{a}}_j, \hat{\mathbf{b}}) \leftarrow (\mathsf{ModUp}(\mathbf{a}_j), \mathsf{ModUp}(\mathbf{b}))$ for $j \in [0,\beta)$ ▷ $\mathcal{R}^{\beta+1}_{PQ_\ell}$
3: **for** i from 0 to $r-1$ **do**
4: $\quad \hat{\mathbf{a}}^{(j)}_{rot} \leftarrow \mathsf{Automorph}(\hat{\mathbf{a}}_j, i\cdot d)$ for $j \in [0,\beta)$
5: $\quad (\hat{\mathbf{u}}, \hat{\mathbf{v}}) \leftarrow \mathsf{KSKInnerProd}(\hat{\mathbf{a}}^{(j)}_{rot}, \mathbf{ksk}_i)$
6: $\quad \hat{\mathbf{b}}_{rot} \leftarrow \mathsf{Automorph}(\hat{\mathbf{b}}, i\cdot d)$
7: $\quad (\mathbf{a}', \mathbf{b}') \mathrel{+}= \mathbf{M}_i \cdot (\hat{\mathbf{u}}, \hat{\mathbf{v}} + \hat{\mathbf{b}}_{rot})$ ▷ DiagInnerProd
8: $[\![\mathbf{m}']\!] \leftarrow (\mathsf{ModDown}(\hat{\mathbf{a}}'), \mathsf{ModDown}(\hat{\mathbf{b}}'))$ ▷ $\mathcal{R}^2_{Q_\ell}$
9: **return** $\mathsf{Rescale}([\![\mathbf{m}']\!])$ ▷ $\mathcal{R}^2_{Q_{\ell-1}}$

Consequently, optimizing on-chip reuse of ct and minimizing off-chip access becomes crucial, especially when working with limited SRAM (e.g., 43 MB on the AMD Alveo U280).

3.1.2 Proposed datapath. To mitigate the on-chip memory requirement and reduce off-chip ct transfer, we propose a novel HLT datapath named Memory Efficient HLT (ME-HLT), shown in Fig. 2. There are several memory-centric optimizations:

Collapsing BSGS: To reduce the number of intermediate ct, we collapse the BSGS into a single rotation loop with r iterations, shown in Algorithm 5. This only generates $(\beta+1)$ intermediate ct with modulus PQ_ℓ in Line 2. Collapsing BSGS also enables us to hoist all Decomp, ModUp and ModDown out of the rotation iterations (Line 1-2 & 8, Algorithm 5), ensuring they are executed only once. Both ModDown and ModUp require computation-heavy NTT/INTT. They also require access to the same coefficient position across different ct limbs during BaseConv. This access pattern is orthogonal to the limb-wise access that reads all coefficients in one limb. Frequent access pattern switch significantly increases off-chip memory access overhead [61]. Therefore, compared to BSGS with double-hoisting where ModDown and ModUp need to execute (r_2+1) times, our method greatly reduces the computational overhead and switching off-chip access patterns during HLT.

Overall, collapsing BSGS reduces the on-chip SRAM requirement and enables hoisting all ModUp and ModDown to reduce off-chip ct transfer overhead. Although collapsing BSGS increases the number of Rotate from $O(\sqrt{r})$ to $O(r)$, this is mitigated by the improved memory efficiency for an SRAM-limited device. Also, we further

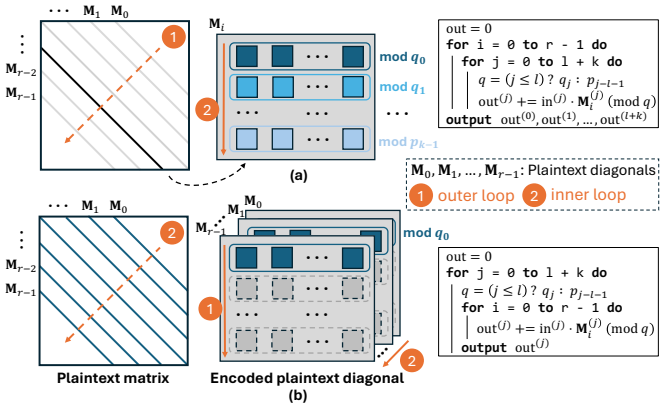

Figure 3: Computation pattern of DiagInnerProd **(e.g., *rdx*=4, *r*=7): (a) Traditional method; (b) Fine-grained method.**

eliminate off-chip ct limb transfer during rotation iterations through a fine-grained datapath introduced as follows.

Fine-grained datapath: The fine-grained datapath is enabled by subroutine fusion based on limb-level computation. *Limb-level* computation refers to a process where one ct limb completes the current subroutine and moves to the next subroutine before the other limbs of the same ct complete the current subroutine. It differs from traditional *ct-level* computation, where all limbs of a ct must complete the current subroutine and then proceed to the next subroutine together. For example, in limb-level computation, it is not necessary to complete Automorph of all ct limbs before starting KSKInnerProd (Line 4-5, Algorithm 5).

As mentioned above, we avoid switching access patterns during rotation iterations by collapsing BSGS. Hence, we can extensively perform limb-level computation by fusing subroutines to increase on-chip ct reuse. The proposed ME-HLT in Fig. 2 fuses subroutines within rotation iterations, including KSKInnerProd, Automorph, and DiagInnerProd, where the off-chip ct transfer is eliminated. Based on Algorithm 5, we further follow the optimization of reordering Automorph and KSKInnerProd in Rotate (Line 4 & 5), proposed in [12]. This is achieved by pre-rotating the **ksk**; it reduces the number of Automorph from $(\beta+1)r$ to $3r$ for each *fftiter*.

In KSKInnerProd, computation is performed between limbs that share the same modulus. Specifically, 2β **ksk** limbs are streamed to compute with β ct limbs (all sharing the same modulus) from β digits and generate two output limbs. These two on-chip output limbs then proceed directly to Automorph without off-chip transfer.

Table 4: Number of limbs of ct polynomials when they are generated during PolyEval **with** $\{m = 5, h = 2\}$.

# of limbs	Chebyshev Poly	Coefficient Poly
ℓ	T_1	-
$\ell - 1$	T_2	-
$\ell - 2$	T_3, T_4	$p_{(2,0)}, \cdots, p_{(2,7)}$
$\ell - 3$	T_8	$p_{(1,0)}, \cdots, p_{(1,3)}$
$\ell - 4$	T_{16}	$p_{(0,0)}, p_{(0,1)}$
$\ell - 5$	-	p

In DiagInnerProd, each encoded pt diagonal contains $(\ell + k + 1)$ limbs, the same number as the limbs of its input ct. Multiplication occurs between the pt limbs and ct limbs that share the same modulus. This enables us to optimize the computation pattern of DiagInnerProd as illustrated in Fig. 3. The traditional method [12] is based on ct-level computation, where all $(\ell + k + 1)$ limbs of the input ct are multiplied with one diagonal before they proceed to the next diagonal. This incurs large memory traffic, as the ct of size $\mathcal{B}_{ct}^{\ell+k+1}$ has to be transferred from off-chip memory for each of the r iterations. In contrast, we reorder the two loops as [59], shown in Fig. 3. This enables limb-level computation where each ct limb is multiplied with the corresponding limb from all r diagonals, before proceeding to the next ct limb. It allows us to fuse DiagInnerProd with its preceding subroutine, Automorph, as shown in Fig. 2. This fusion significantly reduces on-chip memory requirements, as only one result ct limb needs to be stored on-chip.

On-the-fly limb generation: Before performing KSKInnerProd, the number of limbs in each digit is extended from α to $(\ell + k + 1)$ through ModUp. In the traditional method [12], all these required new limbs are generated at once. In ME-HLT shown in Fig 2, ModUp is fused with the subsequent KSKInnerProd. Specifically, each digit generates only one limb at a time and immediately feeds it to KSKInnerProd. For each of the $(\ell + k + 1)$ iterations, the β limbs generated from the β digits have the same modulus, so they can be used to perform the following KSKInnerProd in situ.

Overall, this optimization provides two key memory benefits. Firstly, it significantly reduces the on-chip memory requirement. Only β limbs need to be stored at a time, compared to β digits with total size $\beta \cdot \mathcal{B}_{ct}^{\ell+k+1}$ in the traditional method. Secondly, this enables the fusion of BaseConv, NTT and KSKInnerProd, eliminating the off-chip ct transfer among them. Combined with subroutine fusion from KSKInnerProd to DiagInnerProd, our fine-grained ME-HLT achieves subroutine fusion from BaseConv to DiagInnerProd, greatly improving on-chip ct reuse.

Merging Rescale **into** ModDown**:** Originally, the ModDown in Line 8 of Algorithm 5 reduces the modulus from PQ_ℓ to Q_ℓ, and the Rescale in Line 9 further reduces it to $Q_{\ell-1}$. With the optimization proposed in [1], ModDown directly reduces the modulus to $Q_{\ell-1}$, thereby eliminating the computational cost and off-chip data transfer overhead associated with the separate Rescale.

3.2 Optimized Polynomial Evaluation (OPE)

We propose a novel datapath for polynomial evaluation during bootstrapping by fine-tuning the evaluation circuit. In comparison with

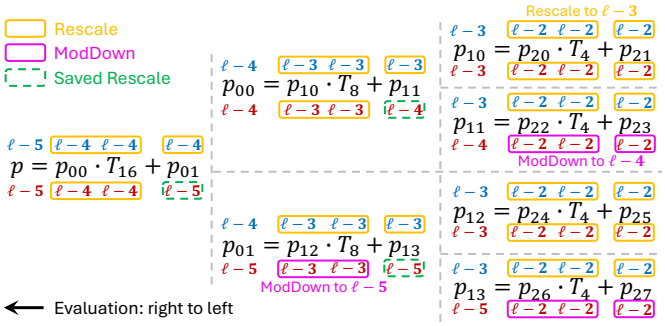

Figure 4: Optimized polynomial evaluation in giant steps. The number of limbs and required Rescale **or** ModDown **operations are shown above the equations (traditional method) and below the equations (our method).**

the ME-HLT targeting SRAM-limited devices, this optimization is purely algorithmic and hardware-agnostic.

To illustrate our method, consider $n = 31, m = 5, h = 2$ as an example, in which case the BSGS Algorithm 3 consumes 5 levels. The Chebyshev polynomials $\{T_2, T_3, T_4, T_8, T_{16}\}$ are calculated iteratively (Line 3-4, Algorithm 3). Each iteration involves a HMult followed by a Rescale. Consequently, the number of limbs for each Chebyshev polynomials progressively decreases, as shown in Table 4.

To evaluate the lowest-level polynomials $\{p_{(2,0)}, \cdots, p_{(2,7)}\}$ (Line 5, Algorithm 3), we need to calculate the linear combinations of Chebyshev polynomials $\{T_0, T_1, T_2, T_3, T_4\}$. As ct-ct operations require operands having the same number of limbs, one ModDown or Rescale is required for each Chebyshev polynomial whose limbs are more than the smallest ($\ell - 2$ in our case). For the same reason, when evaluating upper-level polynomials (Line 6, Algorithm 3), ModDown or Rescale are also required. As introduced in Sec. 2.1, Rescale is a special case of ModDown, both having similar computational costs. Here, we propose an OPE method by replacing multiple Rescale with ModDown operations, illustrated in Fig. 4. On the one hand, it saves multiple Rescale operations (e.g., for $p_{(1,1)}$ and $p_{(1,3)}$). On the other hand, ModDown reduces the polynomial more levels than Rescale, which saves off-chip data traffic. Overall, in the example shown in Fig. 4, our OPE saves 3 Rescale operations and replaces 5 more with ModDown operations.

3.3 Advanced Bootstrapping Algorithm (ABA)

In the traditional bootstrapping as shown in Fig. 1, the input ct for the first HLT (i.e., StC) has $(L + 1 - \ell_{CtS} - \ell_{eval})$ limbs. In contrast, StC is advanced in ABA as in Algorithm 4, and thus its input ct has only $(\ell_{StC}+1)$ limbs. Consequently, the computational cost of StC is reduced by approximately a factor of $\frac{L+1-\ell_{CtS}-\ell_{eval}}{\ell_{StC}+1}$ (e.g., 2.2× with parameter set[2] in [2]), proportional to the number of input limbs. This reduction also scales down the **ksk** size, pt size, and twiddle factor (for NTT/INTT) sizes, as they have the same number of limbs as ct, thereby proportionally decreasing the off-chip memory traffic. Except for StC, all other steps in the traditional bootstrapping have the same computational cost and on-chip memory requirements as in ABA. Overall, advancing StC in ABA significantly reduces the complexity of the first HLT in bootstrapping while incurring

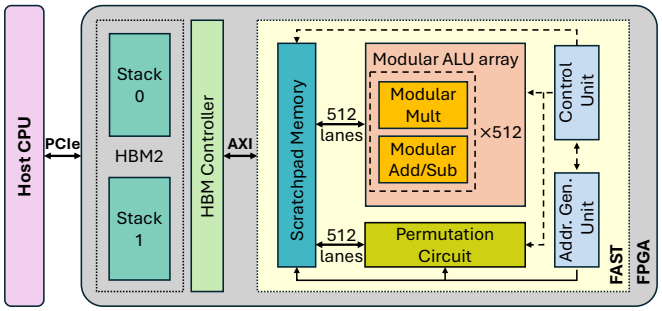

Figure 5: Overall system architecture of FAST

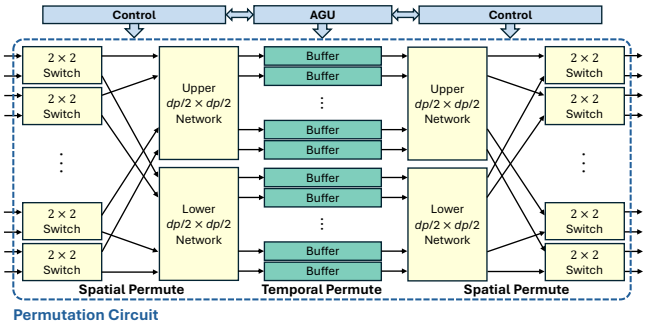

Figure 6: Architecture of permutation circuit with control

only minor overhead for the additional CMult and Conjugate. Our implementation of ABA integrates the proposed ME-HLT (for StC and CtS) and OPE (for PolyEval). Note that both ME-HLT and OPE can also be applied to the traditional bootstrapping algorithm.

4 FHE Accelerator - FAST

4.1 Overall Architecture

The overall system architecture includes the host CPU and FAST, shown in Fig. 5. FAST consists of a modular arithmetic logic unit (ALU) array, scratchpad memory, permutation circuit, HBM, address generation unit (AGU), and a control unit.

The overall dataflow is as follows. First, data are transferred from the host DDR to the HBM of the FPGA acceleration card via the PCIe interface. Next, data, primarily various types of limbs, are loaded from HBM to the scratchpad memory through 32 AXI channels [7]. The accelerator begins computation, interfacing with the HBM via the HBM controller. Results are transmitted back to the host after the kernel execution of a given application is completed.

4.2 Permutation Circuit

HE operations that involve NTT/INTT and Automorph require permutation of limb coefficients. For NTT/INTT, the permutation pattern between two stages follows a fixed, pre-determined stride. The permutation groups every pair of coefficients separated by the stride distance for the butterfly operation. In comparison, the Automorph permutation pattern features varied strides between consecutive coefficients based on the rotation distance and index mapping pattern introduced in Sec. 2.1.

Due to the above complex data access pattern, such operations are hard to process in parallel. Some previous works require a fully connected crossbar with carefully designed memory access scheduling to avoid scratchpad bank conflicts [53, 56]. This is not scalable due to the quadratic growth in number of switches. Moreover, previous SOTA FPGA design [62] utilizes separate permutation circuits for NTT/INTT and Automorph due to the varied permutation patterns. Such a design is not flexible to accommodate various communication patterns and leads to inefficient resource utilization.

In contrast, we design a versatile permutation circuit that supports both NTT/INTT permutation pattern and Automorph permutation pattern, shown in Fig. 6. The circuit is designed based on the streaming permutation network (SPN) [15], which enables parallel permutations of coefficients without relying on the costly crossbar. Our permutation circuit has three subnetworks - two spatial permutation networks and one temporal permutation network. Each spatial network has $\log dp$ stages, using $(dp/2)$ 2×2 switches recursively to create a dp-to-dp connection. The AGU generates buffer addresses and issues read and write commands to dp dual-port buffers to perform the temporal permutation. Each buffer can store N/dp coefficients. Our fully pipelined permutation circuit achieves a throughput of dp, i.e., 512.

The SPN was initially designed for high-throughput permutation given a fixed stride (e.g., NTT/INTT permutation stride [64]). It does not require reconfiguring switches. However, it lacks the support of Automorph index mapping pattern. To enhance its flexibility, we extend the AGU to accommodate the Automorph permutation pattern. The AGU computes the mapped indices of the Automorph and, in coordination with the control unit, generates the control signals for spatial (switch) and temporal (buffer) configurations of the permutation circuit. With this novel AGU, our permutation circuit can support any Automorph index mapping pattern for varying rotation distances without requiring reconfiguration. It should be noted that the control mechanism of our permutation circuit differs from that of the SPN, as configurations are dynamically generated instead of only using a table to store routing information.

More importantly, our permutation circuit supports on-the-fly permutations during the generation of intermediate limbs, enabled by efficient operation scheduling from the control unit and concurrent memory accesses in the scratchpad. Specifically, in each cycle, 512 generated coefficients of a limb are stored in a designated bank for limbs awaiting permutation. In the next cycle, these 512 coefficients are immediately fed into the permutation circuit for processing. Overall, our permutation circuit achieves high throughput, supports a variety of permutations, enables parallel processing with the modular ALU array, and is dynamically controllable at runtime based on the needed permutation.

4.3 Other Hardware Modules

4.3.1 Modular ALU array. The modular ALU array consists of 512 ALUs, enabling FAST to achieve 512 *data parallelism* (dp) with a throughput of 512 coefficients per cycle. Our design achieves maximum parallelism based on the available DSPs in the target FPGA, i.e., Alveo U280. Each fully pipelined ALU consists of one modular multiplier (MM) core and one modular adder (MA) core, taking 11 clock cycles to perform a single scalar modular operation. The MM core is built based on [39], consisting of one full coefficient width

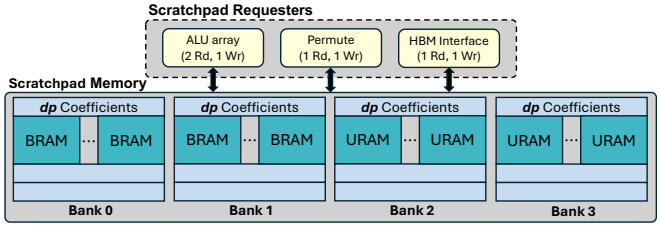

Figure 7: Scratchpad memory architecture and its requesters

integer multiplier and two half coefficient width integer multipliers. We implement the Barret reduction algorithm [30] for modular reduction. The MA core is capable of performing subtraction based on the control signal. The modular ALU can also perform modular multiply-addition/subtraction for butterfly operation in NTT/INTT. Thus, the modular ALU array is shared across various HE operations, handling all necessary modular operations needed in HE.

4.3.2 Scratchpad Memory. We design a scratchpad memory to store polynomials or limbs. Fig. 7 illustrates the architecture of our scratchpad, which includes four memory banks capable of handling four concurrent read requests from the modular ALU array, the permutation circuit, and the HBM interface. Therefore, the on-the-fly permutation mentioned in Sec. 4.2 can be achieved. Two banks are built with BRAM, and the other two are built with URAM. The two BRAM banks are solely dedicated to storing ct with a total capacity of 24 limbs. In contrast, our URAM banks are used to store not only ct limbs but also twiddle factors, pt limbs, and **ksk** limbs, with a total storage capacity of 96 limbs. Each bank has a width of 512 coefficients, ensuring a throughput of dp.

The memory banks are structured in the following manner. For each row of BRAM/URAM blocks, 1024/4096 (×512) coefficients are stored, respectively. These rows of BRAM/URAM blocks are stacked vertically in each bank. Beyond the scratchpad memory, we also utilize multiple register files (RFs) to store pre-computed values, such as powers-of-5, which are used by the AGU for the Automorph permutation.

4.4 Operation Mapping

In this subsection, we briefly explain how the key HE subroutines are executed on FAST.

4.4.1 RNS Basis Conversion. As mentioned in Sec. 3.1.2, BaseConv subroutine requires accessing the same position of coefficients across different limbs. Therefore, ct is accessed in a block-wise manner, rather than processing limb-by-limb (row-by-row). Specifically, the first 512 coefficients of each limb are accessed first, forming one $\ell \times 512$ block. The coefficients of the generated limbs are computed with consecutive MM and MA operations [17]. The generated limbs are then stored in the scratchpad memory banks, and the pre-computed moduli are stored in the RFs.

4.4.2 Number Theoretic Transform. We implement the same data mapping logic for NTT and INTT with the unified Cooley-Tukey algorithm [49]. The modular array functions as $dp/2$ radix-2 butterfly units (BFU), with each BFU processing two limb coefficients and producing two output coefficients. The NTT/INTT datapath consists of $\log N$ stages in total, where N is the number of coefficients in a polynomial. After each stage, the permutation circuit performs fixed permutations, and the specific permutation step for each stage is determined by a routing table in the control logic. Memory access within the temporal network is managed by the AGU. Due to the specialized design of our permutation circuit for on-the-fly permutation, the NTT/INTT datapath archives seamless data streaming between stages. Considering throughput only, the conversion takes N/dp cycles, rather than $\log N \cdot N/dp$.

4.4.3 Automorph. As discussed in Sec. 4.2, the Automorph subroutine is also realized using our permutation circuit similar to NTT/INTT. Unlike the fixed pattern in NTT/INTT, which is controlled by a routing table within the control unit, the AGU calculates the new mapped indices through bit-shifting and AND operations with precomputed powers-of-5. Based on the newly computed indices and switch configuration bits, the coefficients are read from the scratchpad and then permuted using the permutation circuit.

Table 5: FHE parameters used for evaluating FAST

$\log q$	N	L	k	dnum	*fftiter*	λ
32	2^{16}	38	16	3	4	128

5 Evaluation and Discussion

5.1 Experimental Setup

We implement FAST using Verilog HDL and map it on AMD Alveo U280 with Vivado [6] and Vitis 2023.1 [5]. Vitis is used to compile and link the RTL code to produce the binary, which is then executed by the host code. The host and the kernel communicate through PCIe under the XRT environment [45]. The accelerator leverages HBM2 memory to facilitate off-chip data transfers, utilizing 32 channels with up to 460 GB/s.

Our FHE parameter set is shown in Table 5. We adopt the 128-bit security parameter set as in [2], which is the standard security level in real-world applications [43]. We change the coefficient size to 32 bits, which allows us to achieve a dp of 512 with limited DSP resources. Additionally, it leads to the increase of L (the maximum level of ct), reducing the need for frequent bootstrapping. As a result, the number of HMult/PMult that can be performed between two bootstrapping operations increases from 5 to 23 [2].

It is noteworthy that FAST is scalable and not limited to the selected parameter set. This is mainly due to our fine-grained dataflow with subroutine fusion operating on limbs rather than ciphertexts, which significantly reduces on-chip memory requirements for HE operations. By storing intermediate limbs on-chip, FAST remains efficient even for larger parameter sets, e.g., $N = 2^{17}$.

5.2 Performance

We evaluate the performance of FAST and compare it against the SOTA implementations w.r.t. the latency of basic HE operations, bootstrapping, and end-to-end applications. For a fair comparison, we adopt the same 100-bit security parameter set as used in [2, 34] for basic HE operations, while we use our 128-bit security parameter set discussed in Sec. 5.1 for performance comparison of bootstrapping and applications.

Table 6: HE operation latency (*ms*) comparison with SOTA

Work	λ	Freq. (GHz)	HAdd	HMult	PMult	Rotate	Rescale
CPU [34]	98	3.3	28.12	2645	26.22	2579	145
GPU [34]	100	1.2	0.16	2.96	0.14	2.55	0.49
FAB [2]	100	0.3	0.04	1.71	-	1.57	0.19
HEAX [53]	98	0.3	0.24	8.4	0.24	-	-
Poseidon [62]	98	0.45	0.07	3.66	0.08	3.31	0.25
FAST	100	0.3	0.09	1.42	0.08	1.36	0.14

5.2.1 HE Operations. For basic HE operations, the CPU baseline [34] is measured on an Intel Xeon Gold 6234 CPU using a single thread. For the GPU baseline, we use the best performance number reported in the SOTA work [34]. HEAX [53] is a pioneering FPGA implementation of HE that does not support bootstrapping. FAB [2] and Poseidon [62] are two SOTA FPGA designs that support bootstrapping, both using the same acceleration card as this work, i.e., AMD Alveo U280.

Table 6 shows the comparison of latency w.r.t. basic HE operations. Specifically, FAST achieves 1.8-3.5× speedup across all operations compared to GPU, with greater gains over more compute-bound operations, e.g. Rescale. Compared to other FPGA designs, FAST achieves 1.2-5.9× and 1.2-2.4× speedups w.r.t. HMult and Rotate, respectively. These improvements are primarily due to the shared modular ALU array, the versatile permutation circuit that supports efficient on-the-fly permutation, and seamless data streaming between NTT/INTT stages, as described in Sec. 4.3 and Sec. 4.4.

5.2.2 Bootstrapping. As in the literature [2, 34, 38], we evaluate bootstrapping using two metrics: i) latency T_{boot} and ii) *amortized mult time per slot* $T_{\text{HMult},a/s}$. The second metric (smaller is better) is widely used to evaluate bootstrapping performance, given by

$$T_{\text{HMult},a/s} = \frac{T_{boot} + \sum_{i=1}^{\ell} T_{\text{HMult}(i)}}{\ell \cdot (N/2)}, \tag{4}$$

where ℓ denotes the remaining ct levels after bootstrapping, and $N/2$ is the number of slots in the encrypted message, and $T_{\text{HMult}(i)}$ is the latency for HMult at level i. This metric is normalized by the encrypted message size and reflects the bootstrapping performance in real applications, considering the multiplication time between two bootstrapping operations.

As shown in Table 7, FAST can achieve over 1000× speedup over SOTA CPU implementation Lattigo [47, 48] w.r.t. $T_{\text{HMult},a/s}$. Compared with the SOTA GPU work named "100x" [34], FAST performs 6.82× and 8.84× better w.r.t. latency and $T_{\text{HMult},a/s}$, respectively. In comparison to ASIC designs, FAST outperforms F1+ by 420× w.r.t. $T_{\text{HMult},a/s}$. It should be noted that F1+ is a scaled-up version of F1 [54], with its performance estimated in [55]. ARK [36] and CL [55] demonstrate better performance due to much larger SRAM (512 MB v.s. our 43 MB), additional modular ALUs with larger *dp* (8192 v.s. our 512), newer versions of HBM with higher bandwidth (1 TB/s v.s. our 460 GB/s), and higher operating frequencies (1 GHz v.s. our 300 MHz). However, both ARK and CL overlook subroutine fusion for fine-grained limb reuse, which is the key innovation in our ME-HLT approach. Instead, they rely on increased hardware resources to achieve performance gains. FAST achieves an average speedup of 2.3× compared to SOTA FPGA designs, FAB and Poseidon, w.r.t. bootstrapping latency. As for $T_{\text{HMult},a/s}$, FAST outperforms FAB by 5.89×, while Poseidon does not evaluate the performance on it. Overall, FAST has relatively better performance w.r.t. $T_{\text{HMult},a/s}$ due to the lower latency bootstrapping, faster HMult operation, and higher ct levels that lead to less frequent bootstrapping operations.

Table 7: Bootstrapping performance comparison with SOTA

Work	λ	Freq. (GHz)	T_{boot} (*ms*)	↑*	$T_{\text{HMult},a/s}$ (μs)	↑
Lattigo (CPU)	80	3.5	3.9e4	810×	88	1086×
100x (GPU)	106	1.2	328.3	6.82×	0.716	8.84×
F1+ (ASIC)	80	1.0	58.3	1.21×	34	420×
CL (ASIC)	128	1.0	6.33	0.13×	0.017	0.21×
ARK (ASIC)	128	1.0	3.5	0.07×	0.014	0.17×
FAB (FPGA)	128	0.3	92.4	1.92×	0.477	5.89×
Poseidon (FPGA)	128	0.45	127.45	2.65×	-	-
FAST (FPGA)	128	0.3	48.11	-	0.081	-

∗ In this paper, ↑ denotes the speedup achieved by FAST.

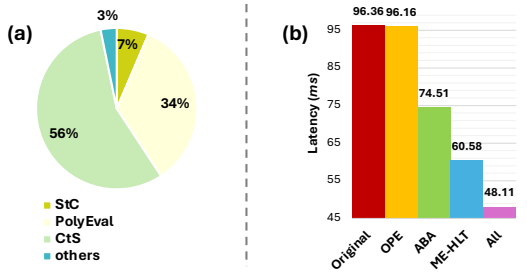

Figure 8: Bootstrapping: (a) Execution time breakdown; (b) Ablation Study

We also evaluate the breakdown of bootstrapping latency, shown in Fig. 8(a). Notably, the first HLT (i.e., StC) accounts for only 7% of the total latency, primarily due to the reduced number of limbs achieved by adopting the ABA, leading to reduced computational and storage requirements. Additionally, we conduct an ablation study on ABA, ME-HLT, and OPE to understand the effects of each optimization. Fig. 8(b) shows the bootstrapping latency after applying the above three optimizations, respectively. The results show that all three optimizations can effectively improve the bootstrapping performance. In particular, ME-HLT has the greatest impact, primarily due to its significant reduction of off-chip memory traffic.

5.2.3 End-to-End Applications. We evaluate FAST on two deep and complex applications: Logistic Regression (LR) training and ResNet-20 (RN20) [31] inference. Bootstrapping needs to be executed periodically during the entire computation, thereby showing the efficiency of FAST on deep evaluations.

The LR training method is based on the HELR algorithm [26, 27], which trains a binary classifier with a subset of MNIST. We adopt the

Table 8: Application performance comparison with SOTA

Work	LR training time/iter. (*s*)	↑	RN20 infer. time/img. (*s*)	↑
Lattigo (CPU)	37.05	1029×	1377	741×
100x (GPU)	0.775	21.5×	-	-
F1+ (ASIC)	0.639	17.8×	2.693	1.45×
BTS (ASIC)	0.028	0.78×	2.020	1.09×
CL (ASIC)	0.015	0.42×	0.321	0.17×
ARK (ASIC)	0.008	0.22×	0.125	0.07×
FAB (FPGA)	0.103	2.86×	-	-
Poseidon (FPGA)	0.073	2.03×	2.661	1.43×
FAST (FPGA)	0.036	-	1.859	-

Table 9: FPGA resource usage of FAST

	DSP	BRAM	URAM	LUT	FF
Modular ALU Array	5632	0	0	421k	901k
Scratchpad Memory	0	3072	768	29k	39k
Permute Circuit	0	512	0	125k	240k
Others	0	0	0	215k	365k
Total	5632 [62%]	3584 [89%]	768 [80%]	790k [61%]	1545k [59%]

same mini-batch training method as in [27] with a batch size of 1024. We train the model for 30 iterations and report the performance as the average training time per iteration. Each iteration consumes 5 ct levels. Our bootstrapping implementation consumes 15 levels. A total of 6 bootstrapping operations are required for training 30 iterations. For RN20, we adopt the CKKS implementation in [43], and evaluate the inference time on a CIFAR-10 [41] RGB image.

Table 8 compares the application performance of FAST with SOTA designs. For LR training, the speedups over CPU (Lattigo [12]) and GPU (100x [34]) are over 1000× and 20×, respectively. Compared to ASICs, FAST achieves 17.8× speedups over F1+ [55]. FAST achieves nearly half the performance of CL [55], while utilizing significantly fewer hardware resources, as discussed in Sec. 5.2.2. FAST outperforms both SOTA FPGA designs, Poseidon [62] and FAB [2], with speedups of 2.03× and 2.86×, respectively. For RN20 inference, FAST achieves speedups of 1.43× and 1.09× over Poseidon [62] and BTS [38], respectively. Our optimizations are primarily aimed at improving bootstrapping performance. Since bootstrapping represents a relatively smaller portion of the overall computation in RN20 inference, the speedups observed for RN20 inference are generally smaller compared to those for LR training.

5.3 Resource Utilization

Table 9 shows the resource utilization of FAST using AMD Alveo U280 [4]. Only the modular array consumes DSP resources, with each modular unit using 11 DSPs. The scratchpad memory comprises two BRAM banks and two URAM banks. We also utilize BRAM resources to build buffers for temporal permutation in the permutation circuit. FAST enhances resource efficiency through its shared modular ALU array and versatile permutation circuit, consuming about 60% of total LUT and FF.

6 Related Work

CPU/GPU: Several FHE libraries [9, 24, 25, 48] have been developed, which support bootstrapping. However, the bootstrapping performance is limited due to the large number of DRAM accesses on the CPU. For example, it takes roughly 8 minutes to bootstrap 128 slots within a ct of degree 2^{16} using [25]. Jung et al. [34] implemented GPU acceleration for various HE operations, including bootstrapping. They proposed kernel fusion to reduce data movement, while certain kernels remain unfused due to on-chip storage constraints. In contrast, we achieve more subroutine fusion in bootstrapping with the available 43 MB SRAM of AMD Alveo U280.

FPGAs: Many FPGA-based HE accelerators [29, 53, 60, 63] are limited to small parameter sets or specific operations. They do not support bootstrapping. Recent advances supporting bootstrapping have demonstrated higher performance than CPU/GPU-based solutions. FAB [2] is the first FPGA design supporting bootstrapping. Poseidon [62] implements the bootstrapping based on [12]. However, existing FPGA-based accelerators overlook the potential for fine-grained ct limb reuse during bootstrapping, so the limited on-chip SRAM constrains the bootstrapping performance. In contrast, FAST achieves efficient bootstrapping performance primarily through leveraging the proposed ME-HLT to improve on-chip ct reuse. Additionally, we extend SPN [15], a scalable FPGA design for high-throughput permutation with a fixed stride, to support diverse permutation patterns in HE. This enables FAST to perform the needed permutations with low overall resource utilization.

ASICs: F1 [54] is the first ASIC design to report performance for CKKS bootstrapping. However, it only supports a small parameter set and single-slot bootstrapping, achieving limited throughput. BTS [38] and CL [55] are two pioneering ASIC designs that report the performance of fully-packed CKKS bootstrapping. ARK [36] builds upon these by improving performance through memory-centric optimizations, such as key-switching key reuse. While ASIC designs can achieve high performance, they require significant hardware resources, e.g., hundreds of megabytes of SRAM.

7 Conclusion

In this paper, we introduced FAST, an HBM-based FPGA accelerator for FHE that achieves superior bootstrapping performance. Through algorithm-architecture co-optimization, we significantly reduced off-chip data transfers. We developed a versatile permutation circuit to handle diverse permutation patterns in FHE. Experimental results show that FAST achieves substantial speedups in both bootstrapping and complex applications compared to SOTA GPU and FPGA designs. In the future, we plan to extend FAST into a multi-FPGA system, targeting deployment in public commercial cloud environments to enable efficient processing of more complex and large-scale FHE applications.

Acknowledgments

This work is supported by the U.S. National Science Foundation (NSF) under grants CSSI-2311870 and SaTC-2104264, as well as the DEVCOM Army Research Office (ARO) under grant W911NF2320186. Equipment and support by AMD AECG are greatly appreciated.
Distribution Statement A: Approved for public release. Distribution is unlimited.

References

[1] Rashmi Agrawal, Leo De Castro, Chiraag Juvekar, Anantha Chandrakasan, Vinod Vaikuntanathan, and Ajay Joshi. 2023. MAD: Memory-Aware Design Techniques for Accelerating Fully Homomorphic Encryption. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 685–697.

[2] Rashmi Agrawal, Leo de Castro, Guowei Yang, Chiraag Juvekar, Rabia Yazicigil, Anantha Chandrakasan, Vinod Vaikuntanathan, and Ajay Joshi. 2023. FAB: An FPGA-based Accelerator for Bootstrappable Fully Homomorphic Encryption. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 882–895. https://doi.org/10.1109/HPCA56546.2023.10070953

[3] AMD. 2020. AMD ACAP Versal HBM-series. https://www.xilinx.com/products/silicon-devices/acap/versal-hbm.html#productAdvantages

[4] AMD. 2023. Alveo U280 Data Center Accelerator Card User Guide. https://docs.amd.com/r/en-US/ug1314-alveo-u280-reconfig-accel/Introduction

[5] AMD. 2023. Vitis Unitifed Software Platform rev:2023.1. https://www.xilinx.com/products/designtools/vitis.html

[6] AMD. 2023. Vivado Design Suite rev:2023.1. https://www.xilinx.com/products/designtools/vivado.html

[7] AMD. 2024. AMBA AXI4 Interface Protocol. https://www.xilinx.com/products/intellectual-property/axi.html#overview

[8] AMD. 2024. AMD Alveo Adaptable Accelerator Cards. https://www.amd.com/en/products/accelerators/alveo.html

[9] Ahmad Al Badawi, Andreea Alexandru, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, Zeyu Liu, Daniele Micciancio, Carlo Pascoe, Yuriy Polyakov, Ian Quah, Saraswathy R.V., Kurt Rohloff, Jonathan Saylor, Dmitriy Suponitsky, Matthew Triplett, Vinod Vaikuntanathan, and Vincent Zucca. 2022. OpenFHE: Open-Source Fully Homomorphic Encryption Library. Cryptology ePrint Archive, Paper 2022/915. https://eprint.iacr.org/2022/915

[10] Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, and Damien Stehlé. 2024. Bootstrapping Bits with CKKS. Cryptology ePrint Archive, Paper 2024/767. https://doi.org/10.1007/978-3-031-58723-8_4 https://eprint.iacr.org/2024/767.

[11] Jean-Claude Bajard, Julien Eynard, M Anwar Hasan, and Vincent Zucca. 2016. A full RNS variant of FV like somewhat homomorphic encryption schemes. In *International Conference on Selected Areas in Cryptography*. Springer, 423–442.

[12] Jean-Philippe Bossuat, Christian Mouchet, Juan Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2021. Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 587–617.

[13] Hao Chen, Ilaria Chillotti, and Yongsoo Song. 2019. Improved bootstrapping for approximate homomorphic encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 34–54.

[14] Hao Chen and Kyoohyung Han. 2018. Homomorphic Lower Digits Removal and Improved FHE Bootstrapping. In *Advances in Cryptology – EUROCRYPT 2018*, Jesper Buus Nielsen and Vincent Rijmen (Eds.). Springer International Publishing, Cham, 315–337.

[15] Ren Chen and Viktor K. Prasanna. 2015. Automatic generation of high throughput energy efficient streaming architectures for arbitrary fixed permutations. In *2015 25th International Conference on Field Programmable Logic and Applications (FPL)*. 1–8. https://doi.org/10.1109/FPL.2015.7293944

[16] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2018. Bootstrapping for Approximate Homomorphic Encryption. In *Advances in Cryptology – EUROCRYPT 2018*, Jesper Buus Nielsen and Vincent Rijmen (Eds.). Springer International Publishing, Cham, 360–384.

[17] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2019. A full RNS variant of approximate homomorphic encryption. In *Selected Areas in Cryptography–SAC 2018: 25th International Conference, Calgary, AB, Canada, August 15–17, 2018, Revised Selected Papers 25*. Springer, 347–368.

[18] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2017. Homomorphic Encryption for Arithmetic of Approximate Numbers. In *Advances in Cryptology – ASIACRYPT 2017*, Tsuyoshi Takagi and Thomas Peyrin (Eds.). Springer International Publishing, Cham, 409–437.

[19] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2017. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology–ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I 23*. Springer, 409–437.

[20] Leo de Castro and Vinod Vaikuntanathan. 2021. Does Fully Homomorphic Encryption Need Compute Acceleration? *arXiv preprint arXiv:2112.06396* (2021).

[21] Shengyu Fan, Xianglong Deng, Zhuoyu Tian, Zhicheng Hu, Liang Chang, Rui Hou, Dan Meng, and Mingzhe Zhang. 2024. Taiyi: A high-performance CKKS accelerator for Practical Fully Homomorphic Encryption. arXiv:2403.10188 [cs.CR] https://arxiv.org/abs/2403.10188

[22] Robin Geelen, Michiel Van Beirendonck, Hilder V. L. Pereira, Brian Huffman, Tynan McAuley, Ben Selfridge, Daniel Wagner, Georgios Dimou, Ingrid Verbauwhede, Frederik Vercauteren, and David W. Archer. 2022. BASALISC: Programmable Hardware Accelerator for BGV Fully Homomorphic Encryption. Cryptology ePrint Archive, Paper 2022/657. https://doi.org/10.46586/tches.v2023.i4.32-57

[23] Craig Gentry. 2009. Fully homomorphic encryption using ideal lattices. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*. 169–178.

[24] Shai Halevi and Victor Shoup. 2015. Bootstrapping for HElib. In *Advances in Cryptology – EUROCRYPT 2015*, Elisabeth Oswald and Marc Fischlin (Eds.). Springer Berlin Heidelberg, 641–670.

[25] Kyoohyung Han, Minki Hhan, and Jung Hee Cheon. 2019. Improved Homomorphic Discrete Fourier Transforms and FHE Bootstrapping. *IEEE Access* 7 (2019), 57361–57370. https://doi.org/10.1109/ACCESS.2019.2913850

[26] Kyoohyung Han, Seungwan Hong, Jung Hee Cheon, and Daejun Park. 2018. Efficient Logistic Regression on Large Encrypted Data. Cryptology ePrint Archive, Paper 2018/662. https://eprint.iacr.org/2018/662

[27] Kyoohyung Han, Seungwan Hong, Jung Hee Cheon, and Daejun Park. 2019. Logistic regression on homomorphic encrypted data at scale. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 9466–9471.

[28] Kyoohyung Han and Dohyeong Ki. 2020. Better bootstrapping for approximate homomorphic encryption. In *Cryptographers' Track at the RSA Conference*. Springer, 364–390.

[29] Mingqin Han, Yilan Zhu, Qian Lou, Zimeng Zhou, Shanqing Guo, and Lei Ju. 2022. coxHE: A software-hardware co-design framework for FPGA acceleration of homomorphic computation. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1353–1358. https://doi.org/10.23919/DATE54114.2022.9774559

[30] Darrel Hankerson, Scott A. Vanstone, and Alfred Menezes. 2004. Guide to Elliptic Curve Cryptography. In *Springer Professional Computing*. https://api.semanticscholar.org/CorpusID:268072435

[31] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Deep Residual Learning for Image Recognition. arXiv:1512.03385 [cs.CV] https://arxiv.org/abs/1512.03385

[32] Intel. 2020. Intel® oneAPI Math Kernel Library. https://www.intel.com/content/www/us/en/docs/onemkl/get-started-guide/2023-0/overview.html

[33] Lei Jiang, Qian Lou, and Nrushad Joshi. 2022. MATCHA: a fast and energy-efficient accelerator for fully homomorphic encryption over the torus. In *Proceedings of the 59th ACM/IEEE Design Automation Conference* (San Francisco, California) *(DAC '22)*. Association for Computing Machinery, New York, NY, USA, 235–240. https://doi.org/10.1145/3489517.3530435

[34] Wonkyung Jung, Sangpyo Kim, Jung Ho Ahn, Jung Hee Cheon, and Younho Lee. 2021. Over 100x Faster Bootstrapping in Fully Homomorphic Encryption through Memory-centric Optimization with GPUs. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2021, 4 (Aug. 2021), 114–148. https://doi.org/10.46586/tches.v2021.i4.114-148

[35] Jongmin Kim, Sangpyo Kim, Jaewan Choi, Jaiyoung Park, Donghwan Kim, and Jung Ho Ahn. 2023. SHARP: A Short-Word Hierarchical Accelerator for Robust and Practical Fully Homomorphic Encryption. In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (Orlando, FL, USA) *(ISCA '23)*. Association for Computing Machinery, New York, NY, USA, Article 18, 15 pages. https://doi.org/10.1145/3579371.3589053

[36] Jongmin Kim, Gwangho Lee, Sangpyo Kim, Gina Sohn, Minsoo Rhu, John Kim, and Jung Ho Ahn. 2022. ARK: Fully Homomorphic Encryption Accelerator with Runtime Data Generation and Inter-Operation Key Reuse. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1237–1254. https://doi.org/10.1109/MICRO56248.2022.00086

[37] Miran Kim and Kristin Lauter. 2015. Private genome analysis through homomorphic encryption. In *BMC medical informatics and decision making*, Vol. 15. Springer, 1–12.

[38] Sangpyo Kim, Jongmin Kim, Michael Jaemin Kim, Wonkyung Jung, John Kim, Minsoo Rhu, and Jung Ho Ahn. 2022. BTS: An Accelerator for Bootstrappable Fully Homomorphic Encryption. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (New York, New York) *(ISCA '22)*. Association for Computing Machinery, New York, NY, USA, 711–725. https://doi.org/10.1145/3470496.3527415

[39] Sunwoong Kim, Keewoo Lee, Wonhee Cho, Jung Hee Cheon, and Rob A. Rutenbar. 2019. FPGA-based Accelerators of Fully Pipelined Modular Multipliers for Homomorphic Encryption. In *2019 International Conference on ReConFigurable Computing and FPGAs (ReConFig)*. 1–8. https://doi.org/10.1109/ReConFig48160.2019.8994793

[40] Seonghak Kim, Minji Park, Jaehyung Kim, Taekyung Kim, and Chohong Min. 2022. EvalRound Algorithm in CKKS Bootstrapping. Cryptology ePrint Archive, Paper 2022/1256. https://eprint.iacr.org/2022/1256 https://eprint.iacr.org/2022/1256.

[41] Alex Krizhevsky, Geoffrey Hinton, et al. 2009. Learning multiple layers of features from tiny images. (2009).

[42] Eunsang Lee, Joon-Woo Lee, Junghyun Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and Woosuk Choi. 2022. Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions. In *International Conference on Machine Learning*. PMLR, 12403–12422.

[43] Joon-Woo Lee, HyungChul Kang, Yongwoo Lee, Woosuk Choi, Jieun Eom, Maxim Deryabin, Eunsang Lee, Junghyun Lee, Donghoon Yoo, Young-Sik Kim, et al. 2022. Privacy-preserving machine learning with fully homomorphic encryption

for deep neural network. *iEEE Access* 10 (2022), 30039–30054.

[44] Zhichuang Liang and Yunlei Zhao. 2022. Number theoretic transform and its applications in lattice-based cryptosystems: A survey. *arXiv preprint arXiv:2211.13546* (2022).

[45] Xilinx Runtime Library. 2023. https://www.xilinx.com/products/designtools/vitis/xrt.html

[46] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. 2010. On ideal lattices and learning with errors over rings. In *Advances in Cryptology–EUROCRYPT 2010: 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, French Riviera, May 30–June 3, 2010. Proceedings 29*. Springer, 1–23.

[47] Christian Vincent Mouchet, Jean-Philippe Bossuat, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2020. Lattigo: A multiparty homomorphic encryption library in go. In *Proceedings of the 8th Workshop on Encrypted Computing and Applied Homomorphic Cryptography*. 64–70.

[48] Christian Vincent Mouchet, Jean-Philippe Bossuat, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2023. Lattigo v5. Online: https://github.com/tuneinsight/lattigo. EPFL-LDS, Tune Insight SA.

[49] Norton and Silberger. 1987. Parallelization and Performance Analysis of the Cooley–Tukey FFT Algorithm for Shared-Memory Architectures. *IEEE Trans. Comput.* 100, 5 (1987), 581–591.

[50] Jaehyoung Park, Dong Seong Kim, and Hyuk Lim. 2020. Privacy-preserving reinforcement learning using homomorphic encryption in cloud computing infrastructures. *IEEE Access* 8 (2020), 203564–203579.

[51] Ran Ran, Wei Wang, Quan Gang, Jieming Yin, Nuo Xu, and Wujie Wen. 2022. CryptoGCN: Fast and scalable homomorphically encrypted graph convolutional network inference. *Advances in Neural information processing systems* 35 (2022), 37676–37689.

[52] Oded Regev. 2009. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)* 56, 6 (2009), 1–40.

[53] M Sadegh Riazi, Kim Laine, Blake Pelton, and Wei Dai. 2020. HEAX: An architecture for computing on encrypted data. In *Proceedings of the twenty-fifth international conference on architectural support for programming languages and operating systems*. 1295–1309.

[54] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Srinivas Devadas, Ronald Dreslinski, Christopher Peikert, and Daniel Sanchez. 2021. F1: A Fast and Programmable Accelerator for Fully Homomorphic Encryption. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) *(MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 238–252. https://doi.org/10.1145/3466752.3480070

[55] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Nathan Manohar, Nicholas Genise, Srinivas Devadas, Karim Eldefrawy, Chris Peikert, and Daniel Sanchez. 2022. CraterLake: a hardware accelerator for efficient unbounded computation on encrypted data. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (New York, New York) *(ISCA '22)*. Association for Computing Machinery, New York, NY, USA, 173–187. https://doi.org/10.1145/3470496.3527393

[56] Sujoy Sinha Roy, Furkan Turan, Kimmo Jarvinen, Frederik Vercauteren, and Ingrid Verbauwhede. 2019. FPGA-Based High-Performance Parallel Architecture for Homomorphic Computing on Encrypted Data. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 387–398. https://doi.org/10.1109/HPCA.2019.00052

[57] Olamide Timothy Tawose, Jun Dai, Lei Yang, and Dongfang Zhao. 2023. Toward efficient homomorphic encryption for outsourced databases through parallel caching. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–23.

[58] Tommy White, Charles Gouert, Chengmo Yang, and Nektarios Georgios Tsoutsos. 2023. Fhe-booster: Accelerating fully homomorphic execution with fine-tuned bootstrapping scheduling. In *2023 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 293–303.

[59] Zhihan Xu, Yang Yang, Rajgopal Kannan, and Viktor K Prasanna. 2024. Bandwidth Efficient Homomorphic Encrypted Discrete Fourier Transform Acceleration on FPGA. In *2024 IEEE 32nd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 1–12.

[60] Yang Yang, Rajgopal Kannan, and Viktor K Prasanna. 2024. A Framework for Generating Accelerators for Homomorphic Encryption Operations on FPGAs. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE, 61–70.

[61] Yang Yang, Weihang Long, Rajgopal Kannan, and Viktor K Prasanna. 2023. FPGA Acceleration of Rotation in Homomorphic Encryption Using Dynamic Data Layout. In *2023 33rd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 174–181.

[62] Yinghao Yang, Huaizhi Zhang, Shengyu Fan, Hang Lu, Mingzhe Zhang, and Xiaowei Li. 2023. Poseidon: Practical Homomorphic Encryption Accelerator. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 870–881. https://doi.org/10.1109/HPCA56546.2023.10070984

[63] Tian Ye, Sanmukh R Kuppannagari, Rajgopal Kannan, and Viktor K Prasanna. 2021. Performance modeling and FPGA acceleration of homomorphic encrypted convolution. In *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 115–121.

[64] Tian Ye, Yang Yang, Sanmukh R Kuppannagari, Rajgopal Kannan, and Viktor K Prasanna. 2021. Fpga acceleration of number theoretic transform. In *High Performance Computing: 36th International Conference, ISC High Performance 2021, Virtual Event, June 24–July 2, 2021, Proceedings 36*. Springer, 98–117.

[65] Songnian Zhang, Suprio Ray, Rongxing Lu, Yunguo Guan, Yandong Zheng, and Jun Shao. 2022. Efficient and privacy-preserving spatial keyword similarity query over encrypted data. *IEEE Transactions on Dependable and Secure Computing* 20, 5 (2022), 3770–3786.

[66] Junwei Zhou, Botian Lei, Huile Lang, Emmanouil Panaousis, Kaitai Liang, and Jianwen Xiang. 2023. Secure genotype imputation using homomorphic encryption. *Journal of information security and applications* 72 (2023), 103386.